

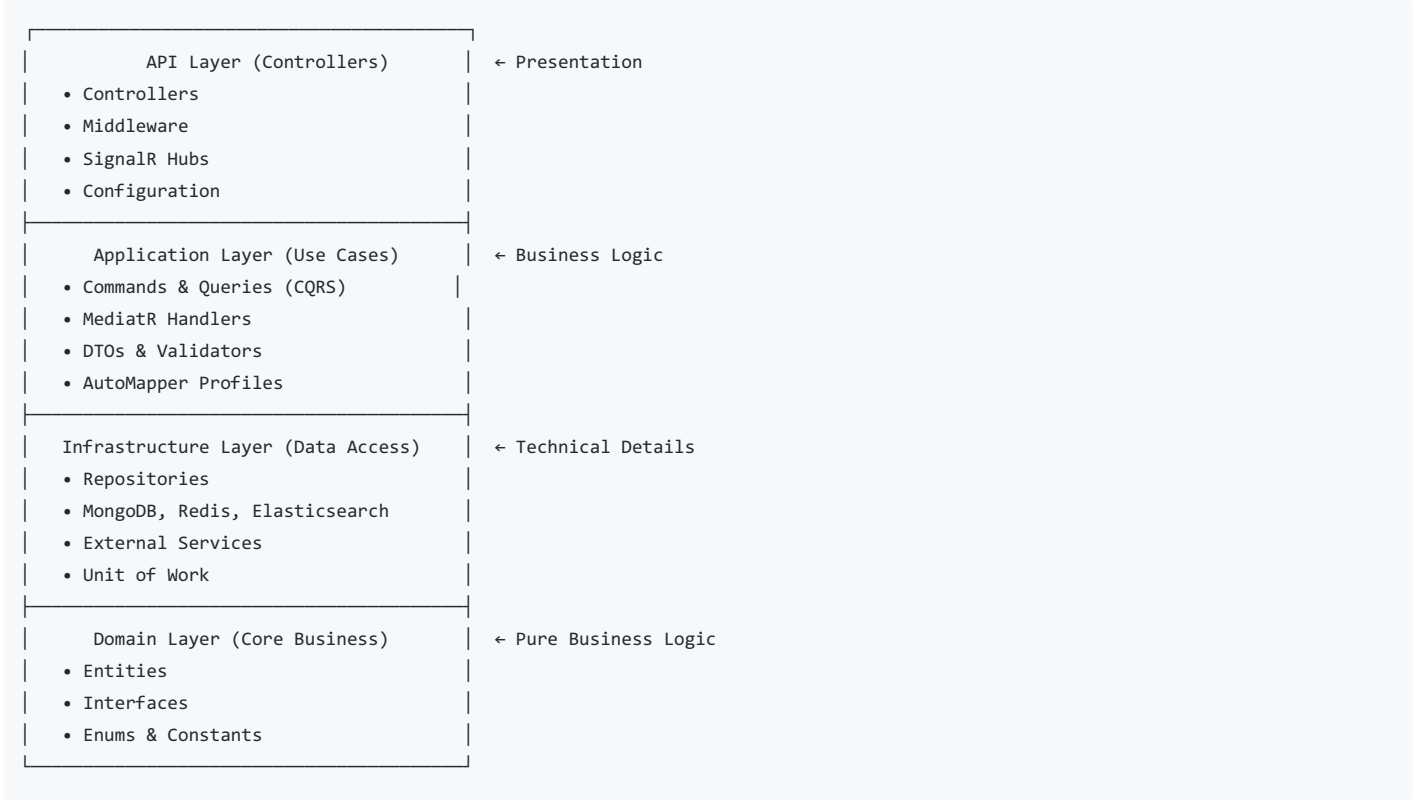
EventUp API Developer Documentation

Bu dokümantasyon EventUp API'sinin UI entegrasyonu için gerekli tüm endpoint'leri, request/response formatlarını ve kullanım örneklerini içerir.

Mimari Yapı

Clean Architecture (Onion Architecture)

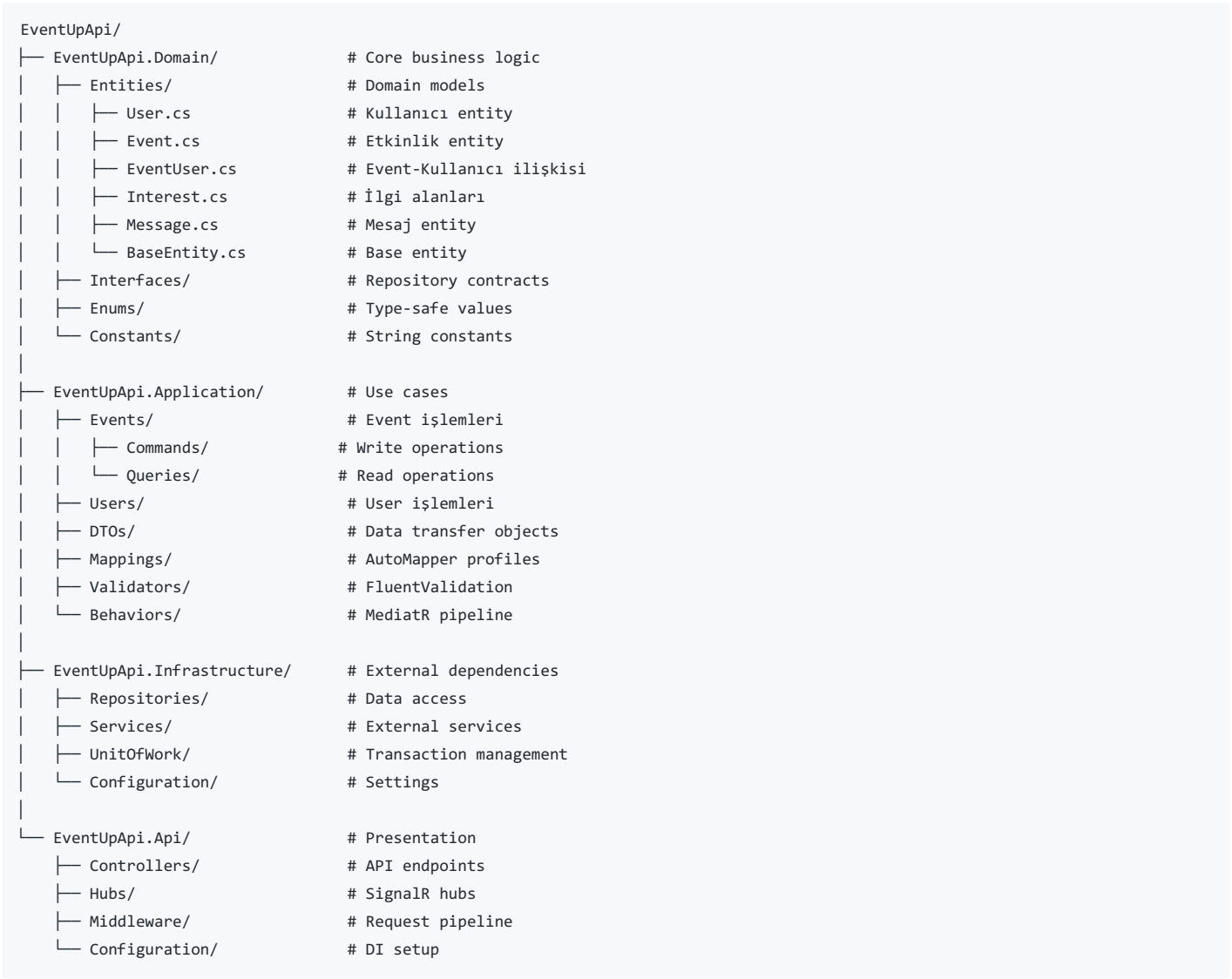
EventUp API, Clean Architecture prensiplerine uygun olarak tasarlanmıştır. Katmanlar arası bağımlılıklar dışa doğru olacak şekilde organize edilmiştir.



Katman Bağımlılıkları

API Layer → Application Layer → Domain Layer
Infrastructure → Application Layer → Domain Layer
Domain Layer → Hiçbir katmana bağımlı değil (Pure)

Proje Yapısı



```
erDiagram
    User {
        string Id
        string UserName
        string Name
        string Surname
        string Email
        string Password
        DateTime LastLoginAt
        DateTime CreatedAt
    }

    Event {
        string Id
        string Title
        string CreatedByUserName
        string Description
        string Location
        DateTime StartDate
        DateTime EndDate
        int Capacity
        string InterestName
        string CoverPhotoUrl
        bool IsLocationConfirmed
        bool IsDateConfirmed
    }

    EventUser {
        string Id
        string EventId
        string UserName
        EventUserStatus Status
        DateTime RequestDate
        DateTime ApprovalDate
        string KickedReason
    }

    Interest {
        string Id
        string Name
    }

    Message {
        string Id
        string MessageGroupId
        string EventId
        string SenderUserName
        string Content
        bool IsRead
    }

    User ||--o{ Event : creates
    Event ||--o{ EventUser : has_participants
    User ||--o{ EventUser : participates
    Interest ||--o{ Event : categorizes
```

Teknoloji Stack

Katman	Teknoloji	Versiyon	Amaç
API	.NET Core	8.0	Web API Framework
Application	MediatR	13.0.0	CQRS Implementation

Application Katman	AutoMapper Teknoloji	12.0.1 Versiyon	Object Mapping Amaç
Application	FluentValidation	11.9.0	Input Validation
Infrastructure	MongoDB	6.0	Primary Database
Infrastructure	Redis	7.0	Distributed Cache
Infrastructure	Elasticsearch	8.11	Log Aggregation
API	SignalR	8.0	Real-time Messaging
API	JWT Bearer	8.0.4	Authentication

Request Pipeline ve Middleware

API request'leri aşağıdaki sırayla işlenir:

- CORS
 - Response Compression & Caching
 - Rate Limiting
 - Serilog Request Logging
 - Authentication & Authorization
 - Global Exception Handling
 - Controllers
 - SignalR Hubs (WebSocket requests)

CQRS Pattern Implementation

Commands (Write Operations):

- Event creation, update, deletion
- User registration, profile updates
- Event participation management

Queries (Read Operations):

- Event filtering and search
- User information retrieval
- Statistics and reporting

Data Flow

- Client Request → API Controller
 - Controller → MediatR → Handler (Command/Query)
 - Handler → Repository (via Unit of Work)
 - Repository → MongoDB/Redis
 - Response ← Repository ← Handler ← Controller ← Client

Caching Strategy

- Redis:** Interest listesi, user sessions
- Response Caching:** Static data, filtered events
- In-Memory:** Frequently accessed data

Authentication Flow

- User Login → JWT Token Generation
 - Token included in Authorization header
 - JWT Middleware validates token
 - User context extracted from token
 - Authorization checks performed

Base URL

https://your-domain.com/api

Authentication

API, JWT (JSON Web Token) tabanlı authentication kullanır. Çoğu endpoint için **Authorization** header'ında Bearer token gereklidir.

Authorization: Bearer <your-jwt-token>

Response Format

Tüm API response'ları aşağıdaki standart formatı kullanır:

```
{
  "success": true,
  "data": { /* response data */ },
  "message": "Operation successful",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

Error response formatı:

```
{
  "success": false,
  "data": null,
  "message": "Error message",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1. Authentication Endpoints

1.1 Register User

Endpoint: POST /api/auth/register **Authentication:** Not required

Request Body:

```
{
  "userName": "johndoe",
  "name": "John",
  "surname": "Doe",
  "email": "john@example.com",
  "password": "SecurePassword123!"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "user123",
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "lastLoginAt": null,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "User registered successfully",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1.2 Login User

Endpoint: POST /api/auth/login **Authentication:** Not required

Request Body:

```
{
  "userNameOrEmail": "johndoe",
  "password": "SecurePassword123!"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "user123",
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Login successful",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

1.3 Get Current User

Endpoint: GET /api/auth/me **Authentication:** Required

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Current user information retrieved successfully",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2. Event Endpoints

2.1 Create Event

Endpoint: POST /api/v1.0/eventowner/create **Authentication:** Required

Request Body:

```
{
  "title": "Tech Meetup 2024",
  "description": "Great tech event for developers",
  "location": "Istanbul, Turkey",
  "startDate": "2024-02-15T18:00:00Z",
  "endDate": "2024-02-15T22:00:00Z",
  "capacity": 50,
  "interestName": "Technology",
  "coverPhotoUrl": "https://example.com/photo.jpg",
  "isLocationConfirmed": true,
  "isDateConfirmed": true
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Tech Meetup 2024",
    "createdByUserName": "johndoe",
    "description": "Great tech event for developers",
    "location": "Istanbul, Turkey",
    "startDate": "2024-02-15T18:00:00Z",
    "endDate": "2024-02-15T22:00:00Z",
    "capacity": 50,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Event başarıyla oluşturuldu",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.2 Get Event by ID

Endpoint: POST /api/v1.0/eventcommon/get-by-id **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Tech Meetup 2024",
    "createdByUsername": "johndoe",
    "description": "Great tech event for developers",
    "location": "Istanbul, Turkey",
    "startDate": "2024-02-15T18:00:00Z",
    "endDate": "2024-02-15T22:00:00Z",
    "capacity": 50,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Event başarıyla getirildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.3 Get Filtered Events

Endpoint: POST /api/v1.0/eventcommon/filtered **Authentication:** Required

Request Body:

```
{
  "search": "tech",
  "location": "Istanbul"
}
```

Response (Success - 200):


```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Filtrelenmiş etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

2.4 Update Event

Endpoint: PUT /api/v1.0/eventowner/update **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "title": "Updated Tech Meetup 2024",
  "description": "Updated description",
  "location": "Ankara, Turkey",
  "startDate": "2024-02-20T18:00:00Z",
  "endDate": "2024-02-20T22:00:00Z",
  "capacity": 75,
  "interestName": "Technology",
  "coverPhotoUrl": "https://example.com/new-photo.jpg",
  "isLocationConfirmed": true,
  "isDateConfirmed": true
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "eventId": "event123",
    "title": "Updated Tech Meetup 2024",
    "createdByUserName": "johndoe",
    "description": "Updated description",
    "location": "Ankara, Turkey",
    "startDate": "2024-02-20T18:00:00Z",
    "endDate": "2024-02-20T22:00:00Z",
    "capacity": 75,
    "interestName": "Technology",
    "coverPhotoUrl": "https://example.com/new-photo.jpg",
    "isLocationConfirmed": true,
    "isDateConfirmed": true,
    "createdAt": "2024-01-15T10:30:00Z",
    "updatedAt": "2024-01-15T11:00:00Z"
  },
  "message": "Event başarıyla güncellendi",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

2.5 Delete Event

Endpoint: DELETE /api/v1.0/eventowner/delete **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "message": "Event başarıyla silindi"
  },
  "message": "Event başarıyla silindi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

3. Event Participation Endpoints

3.1 Join Event

Endpoint: POST /api/v1.0/eventparticipant/join **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Pending",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": null,
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Etkinliğe başarıyla katılım talebi gönderildi",
  "timestamp": "2024-01-15T10:30:00Z"
}
```

3.2 Approve Participant

Endpoint: PUT /api/v1.0/eventowner/approve-participant **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Approved",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": "2024-01-15T11:00:00Z",
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Katılımcı başarıyla onaylandı",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

3.3 Reject Participant

Endpoint: PUT /api/v1.0/eventowner/reject-participant **Authentication:** Required

Request Body:

```
{
  "eventId": "event123",
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "eventuser123",
    "eventId": "event123",
    "userName": "johndoe",
    "status": "Rejected",
    "requestDate": "2024-01-15T10:30:00Z",
    "approvalDate": "2024-01-15T11:00:00Z",
    "kickedDate": null,
    "kickedReason": null
  },
  "message": "Katılımcı başarıyla reddedildi",
  "timestamp": "2024-01-15T11:00:00Z"
}
```

3.4 Leave Event

Endpoint: DELETE /api/v1.0/eventparticipant/leave **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "message": "Etkinlikten başarıyla ayrıldınız"
  },
  "message": "Etkinlikten başarıyla ayrıldınız",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

3.5 Get Approved Participants

Endpoint: POST /api/v1.0/eventcommon/approved-participants **Authentication:** Required

Request Body:

```
{
  "eventId": "event123"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "id": "eventuser123",
      "eventId": "event123",
      "userName": "johndoe",
      "status": "Approved",
      "requestDate": "2024-01-15T10:30:00Z",
      "approvalDate": "2024-01-15T11:00:00Z",
      "kickedDate": null,
      "kickedReason": null
    }
  ],
  "message": "Onaylanmış katılımcılar başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4. User Endpoints

4.1 Get User Basic Info

Endpoint: POST /api/v1.0/user/basic-info **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "userName": "johndoe",
    "name": "John",
    "surname": "Doe",
    "email": "john@example.com",
    "lastLoginAt": "2024-01-15T10:30:00Z",
    "createdAt": "2024-01-15T08:00:00Z",
    "updatedAt": "2024-01-15T10:30:00Z"
  },
  "message": "Kullanıcı temel bilgileri başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.2 Get User Created Events

Endpoint: POST /api/v1.0/eventcommon/created-by-user **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Kullanıcının oluşturduğu etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.3 Get User Participated Events

Endpoint: POST /api/v1.0/eventcommon/participated-events **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event456",
      "title": "Past Event",
      "createdByUserName": "janedoe",
      "description": "Event that already happened",
      "location": "Ankara, Turkey",
      "startDate": "2023-12-15T18:00:00Z",
      "endDate": "2023-12-15T22:00:00Z",
      "capacity": 30,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/past-photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2023-12-10T10:30:00Z",
      "updatedAt": "2023-12-10T10:30:00Z"
    }
  ],
  "message": "Kullanıcının katıldığı etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

4.4 Get User Upcoming Events

Endpoint: POST /api/v1.0/eventcommon/upcoming-events **Authentication:** Required

Request Body:

```
{
  "userName": "johndoe"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "eventId": "event123",
      "title": "Tech Meetup 2024",
      "createdByUserName": "johndoe",
      "description": "Great tech event for developers",
      "location": "Istanbul, Turkey",
      "startDate": "2024-02-15T18:00:00Z",
      "endDate": "2024-02-15T22:00:00Z",
      "capacity": 50,
      "interestName": "Technology",
      "coverPhotoUrl": "https://example.com/photo.jpg",
      "isLocationConfirmed": true,
      "isDateConfirmed": true,
      "createdAt": "2024-01-15T10:30:00Z",
      "updatedAt": "2024-01-15T10:30:00Z"
    }
  ],
  "message": "Kullanıcının katılacağı etkinlikler başarıyla getirildi",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

5. Interest Endpoints

5.1 Get All Interests

Endpoint: GET /api/v1.0/interest **Authentication:** Required

Response (Success - 200):

```
{
  "success": true,
  "data": [
    {
      "id": "interest1",
      "name": "Technology",
      "description": "Tech related events"
    },
    {
      "id": "interest2",
      "name": "Sports",
      "description": "Sports and fitness events"
    }
  ],
  "message": "Interests retrieved successfully",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

5.2 Create Interest

Endpoint: POST /api/v1.0/interest **Authentication:** Required

Request Body:

```
{
  "name": "Art",
  "description": "Art and culture events"
}
```

Response (Success - 200):

```
{
  "success": true,
  "data": {
    "id": "interest3",
    "name": "Art",
    "description": "Art and culture events"
  },
  "message": "Interest başarıyla oluşturuldu",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

6. Error Handling

Common HTTP Status Codes:

- 200 OK - Success
- 400 Bad Request - Invalid request data
- 401 Unauthorized - Missing or invalid authentication
- 404 Not Found - Resource not found
- 500 Internal Server Error - Server error

Error Response Example:

```
{
  "success": false,
  "data": null,
  "message": "Event not found",
  "timestamp": "2024-01-15T11:30:00Z"
}
```

7. Rate Limiting

API rate limiting kullanır. Request limitleri aşılması durumunda 429 Too Many Requests response döner.

8. SignalR Real-time Communication

Message Hub Connection

WebSocket hub endpoint: /messageHub

Connection URL:

```
wss://your-domain.com/messageHub?access_token=<jwt-token>
```

JavaScript Example:


```
const connection = new signalR.HubConnectionBuilder()
  .withUrl("/messageHub", {
    accessTokenFactory: () => {
      return localStorage.getItem('token');
    }
  })
  .build();

connection.start().then(() => {
  console.log("Connected to message hub");
});
```

9. Testing

Health Check Endpoints:

1. **Simple Health Check:**

```
GET /health/ready
```

2. **Detailed Health Check:**

```
GET /health/live
```

3. **Full Health Check:**

```
GET /health
```

10. UI Integration Notes

Token Management:

1. Login/Register sonrası `token` field'ını `localStorage`'da saklayın
2. Her API request'inde `Authorization: Bearer <token>` header'ını ekleyin
3. Token expire olduğunda kullanıcıyı login sayfasına yönlendirin

Error Handling:

1. `success: false` olan response'ları handle edin
2. Network error'ları için retry mechanism implement edin
3. Loading state'leri gösterin

Real-time Updates:

1. SignalR connection kurarak real-time bildirimler alın
2. Connection lost durumlarını handle edin
3. Automatic reconnection implement edin

Bu dokümantasyon EventUp API'sinin ana endpoint'lerini kapsar. Daha detaylı bilgi için API'nin Swagger dokümantasyonunu kullanabilirsiniz (Development ortamında `/swagger` endpoint'i).